

0.98. Une lettre qui tire — la place du A dans deux variables

Le 0.97, et les nombres 2 et 8, la place du A, sont rangés dans `ax` et `ay`. Le tir part de `ax + 1`, sur la ligne `ay` : change `ax` ou `ay`, et le A et son tir bougent ensemble.

1. Ce qu'il faut comprendre

C'est le 0.97, avec une chose en plus : la place du A dans deux variables. À l'écran, rien ne change : le A en (2, 8), le tir qui part à sa droite. Mais le programme est prêt pour le 0.99, où le A va bouger.

Le problème : au 0.97, la place du A est écrite **cinq fois** : `2` dans `poser()`, `3` pour le départ du tir (`2 + 1`), et `8` dans `poser()`, `texte()` et `effacer()`. Pour déplacer le A, il faudrait changer tous ces nombres, sans en oublier un.

Ce qui est nouveau ici : `ax` et `ay`. La colonne et la ligne du A, rangées **une fois**, en haut du programme. Partout où il y avait `2`, il y a `ax` ; partout où il y avait `8`, il y a `ay`.

Le départ du tir est CALCULÉ : `tx = ax + 1`. Avec `ax = 2`, ça donne `2 + 1 = 3`, comme avant. Mais si le A est ailleurs, le tir part toujours **juste à côté de lui**.

Déroulons : `ax = 2`, `ay = 8` : le A en (2, 8). Bouton A : `vole = 1`, `tx = 2 + 1 = 3`, le « . » en (3, 8). Puis (4, 8), (5, 8)... (19, 8), et il disparaît. Le A, lui, n'a pas bougé.

Essaie : écris `uint8_t ax = 0;` et `uint8_t ay = 3;` : le A est en (0, 3), et le tir part de (1, 3). Tu n'as changé que deux nombres.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **543 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          63 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur()  124 o ← main() — une fonction fournie par le langage
main()             135 o ← démarrage — ta fonction
VBlank              8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
```

AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte	– attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), texte(), effacer()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
CopierTuiles	40 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	32 o	← CopierTuiles	– 2 lettres sur 43
"."	1 o	← texte()	– un texte gravé
"."	1 o	← texte()	– un texte gravé

Lettres gravées : A.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 2	36 o sur place, dans main()	+ AdresseCase (10 o), EcrireTexte (10 o)
effacer()	× 1	15 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 1	13 o sur place, dans main()	+ AdresseCase (10 o)
bouton()	× 1	5 o sur place, dans main()	+ LireManette (37 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()	

Tes fonctions appelées :

chaque_minuteur()	× 1	10 o sur place, dans main()
-------------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : la place du A dans deux variables ----
// Le 0.97, et les nombres 2 et 8 (la place du A) sont rangés dans deux
// variables, ax et ay. Le tir part de ax + 1, sur la ligne ay.
// Change ax ou ay : le A ET le tir changent de place ensemble.

#include <poser>           // pose une tuile sur une case du fond
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
#include <bouton>           // lit un bouton de la manette
#include <texte>            // écrit un texte à l'écran
#include <chaque>           // répond 1 toutes les n millisecondes
#include <effacer>         // efface des cases, ou tout le fond
```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;           // NOUVEAU : le A : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...et sa ligne. Il est en (2, 8), comme avant.
```

4. UNE VARIABLE — TX

```
uint8_t tx = 0;           // la colonne du tir
```

5. UNE VARIABLE — VOLE

```
uint8_t vole = 0;         // 1 = le tir est en vol ; 0 = pas de tir
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(ax, ay, ALPHABET[0]);          // le A, à sa place : (ax, ay)

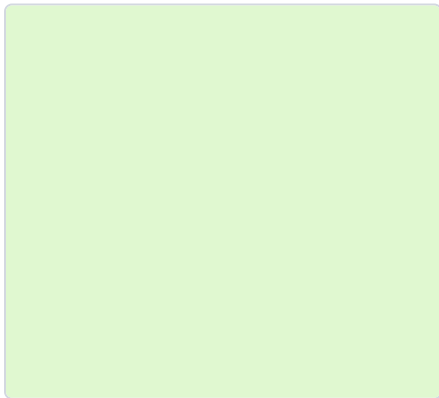
    while (true) {
        image();

        // ---- A lance le tir (le 0.97) ----
        if (bouton(A) && vole == 0) {
            vole = 1;
            tx = ax + 1;                  // NOUVEAU : JUSTE À DROITE du A, calculé :
                                         //   ax vaut 2, donc tx vaut 2 + 1 = 3
            texte(tx, ay, ".");          // sur la ligne du A
        }

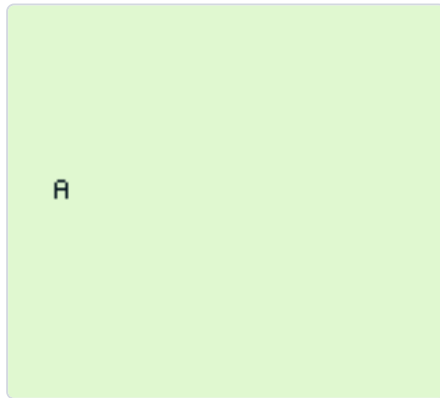
        // ---- le tir avance, sur la ligne du A ----
        if (chaque(100)) {
            if (vole == 1) {
                effacer(tx, ay, 1);
                tx = tx + 1;
                if (tx < 20) {
                    texte(tx, ay, ".");
                } else {
                    vole = 0;
                }
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



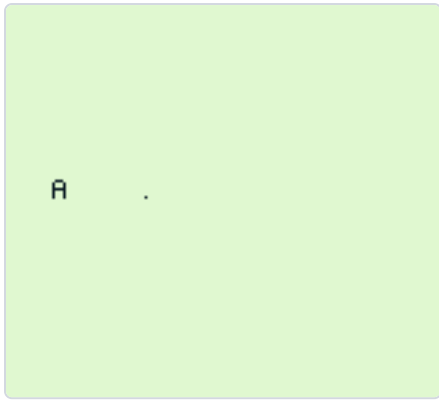
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Comme au 0.97 : un A en (2, 8), et le bouton A lance un « . » juste à sa droite, qui file jusqu'au bord ; le A reste à sa place.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A est en $(ax, ay) = (2, 8)$
- ✓ A : le tir part en $ax + 1 = 3$, juste à sa droite
- ✓ le tir a disparu au bord, le A est toujours là

Ensuite : **0.98.1. Une lettre qui tire — plusieurs tirs, depuis ax et ay** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++